Working with Modern HTML

Live Server, metadata, and HTML text structure — In-Class Discovery Worksheet

Name: _____ Section: _____ Date: _____

How to use this sheet: Keep **VS Code** and your **browser** open side by side. For every task, **attempt and write your answer *before* we discuss it**.

Part 1 — HTML Setup

TRY THIS · VS CODE

In the last session we wrote the HTML boilerplate. Open the `Full_Stack_Sessions` folder again in **Visual Studio Code**. Now create a new folder `2_HTML`, and inside it a new file named `index.html`.

Note: You made changes on the GitHub repo yesterday, which created a `.github` folder on the **remote** — but that folder is not yet on your local machine. So you need to **pull** those changes down:

```
git pull
```

Q1. Can you think of the reason behind keeping the file name as `index.html`?

TRY THIS · VS CODE

Write the **boilerplate code** inside the file using the `!` mark. Then, in the body, write **Hello** using a heading tag and **your name** using a paragraph tag. It should look like this:

```
<body>
    <h1>Hello!</h1>
    <p>I am Likhilesh.</p>
</body>
```

Q2. How will you see the **output** of this file now?

TRY THIS · VS CODE + BROWSER

After you see the output webpage of this HTML file, make an edit to the `h1` tag and **save**:

```
<body>
    <h1>Hello all...</h1>
    <p>I am Likhilesh.</p>
</body>
```

Q3. Do you see the changes on the webpage now? Do you need to **refresh** to see the changes?

TRY THIS · LIVE SERVER

To resolve this refresh issue, we will use an extension called **Live Server**:

1. Click the **Extensions** icon on the left side toolbar.
2. Search for **"live server"**; on the first suggestion (by **Ritwick Dey**), click **Install**.
3. Come back to the **Explorer view** (the default sidebar with all your files).
4. Open `2_HTML` and left-click **once** on `index.html`.
5. Now **right-click** `index.html` and choose the first option, **"Open with Live Server"**.

In future, we will preview our HTML webpages like this.

Q4. Try making some changes to `h1` or `p` and see if they now **auto-update** on the webpage. What is the change you are able to see compared to the **earlier way** of opening an HTML webpage?

Part 2 — The <head>s Up!

Q5. What do you call the **icon** that appears beside the tab name on every browser tab?

TRY THIS · FAVICON

Do a Google search to find the code to add a **favicon** to HTML. You will get a result something like this:

```
<link rel="icon" type="image/x-icon" href="/favicon.ico">
```

Here, `href` is an **attribute** that tells the browser the exact location (URL or file path) of the icon file you want to load. Copy any image and paste it into `2_HTML`. Assuming its name is `something.jpg`, your `href` would look like:

```
<link rel="icon" type="image/x-icon" href="./something.jpg">
```

Q6. Can you now figure out a way to change the **text** that is visible after the icon on the tab?

Q7. We added a `<link>` for a favicon and edited the `<title>` of the document, but **nothing changed on the webpage** itself. Write your thoughts on the content present in the `<head>` tag.

Q8. Yesterday you shared your webpage link with a friend on **WhatsApp** — did it show any **preview**, like other websites do? If yes, which **fields** from your HTML do you think were used to show that preview?

ACTIVITY · OPEN GRAPH + GIT

Do a Google search about **"Open Graph meta tags"** and add `title`, `image` and `description` meta tags inside the `<head>`. Now **add**, **commit** and **push** these changes to GitHub. After you push, go to the **Actions** tab — you should see a new deployment listed there. Then share the link with your friend on WhatsApp once again. Which link will you share?

```
<link>/2_HTML
```

Q9. Do `<link>/2_HTML` and `<link>/2_HTML/index.html` open the **same page**? Write your thoughts here.

Part 3 — <body> Building

TRY THIS · HEADINGS, P & SPAN

Inside the `<body>`, use the `h1`, `h2`, … , `h6` elements. Then use a `p` element, and finally a `span` element.

```
<body>
    <h1>Some text</h1>
    <h2>Some other text</h2>
    ...
    <h6>Some text</h6>
    <p>Some other text</p>
    <span>Some text</span>
</body>
```

Q10. See the output on the webpage and **compare** these tags. Write down your observation.

TRY THIS · TEXT FORMATTING

Now let's do some text formatting inside a `p` tag. Add `<b></b>`, `<i></i>` and `<u></u>` tags somewhere in your code and see the output. For example:

```
<p>I am <b>Likhilesh</b>. I <i>live</i> in <u>Delhi</u></p>
```

Q11. Write your observation about the `<b>`, `<i>` and `<u>` tags.

Q12. Do they work inside `h1` / `h2` / ... / `span`?

TRY THIS · SEMANTIC FORMATTING

Add one more paragraph at the end of the body. Use `<strong></strong>`, `<em></em>` and `<ins></ins>` tags somewhere and see the output:

```
<p>I am <strong>Likhilesh</strong>. I <em>live</em> in <ins>Delhi</ins></p>
```

Q13. Write your observation about `<strong>`, `<em>` and `<ins>`. Compare them with the `<b>`, `<i>` and `<u>` tags.

Concept — Semantic vs. Non-semantic tags. Semantic tags clearly describe their meaning to both the browser and the developer. They explicitly identify the *type* of content they contain, which improves **SEO** and **accessibility**. Non-semantic tags are generic containers that convey no information about their contents.
Think of semantic tags as **labeled boxes** and non-semantic tags as **plain brown boxes**. Major benefits of semantic tags:

- Clear content hierarchy & priority
- Machine-readable context
- Boosted user-experience signals — screen readers & assistive technologies

Q14. Why would anyone use **non-semantic** tags if semantic tags do everything? *(Hint: think about why unlabeled brown boxes are still used in industry.)*

Q15. Make a list of **semantic** elements and **non-semantic** elements.

```
Semantic elements              Non-semantic elements
------------------------------ ------------------------------

_______________________________ _______________________________
_______________________________ _______________________________
_______________________________ _______________________________
_______________________________ _______________________________
_______________________________ _______________________________
```

Q16. If we keep **two paragraph tags on the same line** in the code, do they appear on the same line on the webpage?

Q17. If we use two bold tags or two strong tags inside a paragraph tag, and keep both tags on **separate lines**, what is your observation? For example:

```
<p>
    Hello
    <b>friend</b>,
    how are
    <strong>you</strong>
    ?
</p>
```

Concept — Whitespace collapse. In HTML, extra spaces and newlines in your source code are automatically squashed into a **single space**. This means you need specific tags or character entities to force real layout gaps.

Q18. Try using the `<br>` and `<hr>` elements (you don't need to write their closing tags). Write your inference on them.

Concept — Block vs. Inline. When structuring content, the browser categorizes elements into two kinds. **Block elements** always start on a fresh line and stretch sideways to occupy the full width of their container. **Inline elements** sit side-by-side on the same line.

Q19. Can you now answer **why** `h1`, `h2`, ... , `h6` and `p` come on separate lines?

Q20. Make a list of **block** elements and **inline** elements.

```
Block elements                 Inline elements
------------------------------ ------------------------------
_______________________________ _______________________________
_______________________________ _______________________________
_______________________________ _______________________________
_______________________________ _______________________________
```

ACTIVITY · RECREATE THIS OUTPUT

Write the HTML code to produce the output shown below. **Hint:** use the `p`, `strong`, `em`, `ins`, `sup`, `sub` and `hr` tags.

Chemistry Fact: The chemical formula for water is H_2O.

Physics Fact: Einstein's famous equation is $E = mc^2$.

<u>Important Note:</u> Always remember to close your HTML tags!

CLASS WRAP-UP · GIT

Push all of this code to GitHub.

```
git add .
git commit –m "Working with modern HTML"
git push
```